

AUFRAG C3 · DEPLOYMENT LB1

Cloud-Plattform Deployment auf Azure

Mehmet Ali Gür · HF Informatik · GIBB Berufsfachschule Bern · Mai 2026

1 Umgesetzt und Plattform-Wahl

Im Rahmen des Auftrags C3 wurde eine Node.js-Anwendung (Express + PostgreSQL) auf Microsoft Azure deployt und unter einer öffentlichen HTTPS-URL erreichbar gemacht. Die Anwendung ist eine Notiz-API mit CRUD-Funktionalität, die Notizen in einer Managed PostgreSQL-Datenbank persistiert. Sie erfüllt damit die Anforderung an eine funktionale Backend-Logik — eine reine Statik-Seite genügt dem Auftrag nicht.

Die Anwendung ist live unter <https://mali-notiz-api-c3.azurewebsites.net> erreichbar.

Gewählte Plattform: Microsoft Azure

Als Cloud-Plattform wurde Azure mit zwei Diensten gewählt — App Service (Linux) für die Anwendung und Azure Database for PostgreSQL Flexible Server als Managed Database. Diese Kombination wurde aus den folgenden Gründen bevorzugt:

Managed Hosting — App Service übernimmt OS-Updates, TLS-Zertifikate und Skalierung. Kein VM-Management erforderlich.

Native CI/CD-Integration — Die offizielle `azure/webapps-deploy@v3` GitHub Action ermöglicht reproduzierbares Deployment mit Publish-Profile-Authentifizierung.

Persistenter Managed-Speicher — PostgreSQL Flexible Server in derselben Resource Group — Daten überstehen Neustarts und Redeploys vollständig.

Plattform-Logging — Strukturierte JSON-Logs (Pino) sind im Azure Log Stream direkt einsehbar — kein zusätzliches Tooling nötig.

Im Studenten-Abo (Azure for Students) sind App Service F1 (kostenlos) und PostgreSQL Flexible Server B1ms (750 Free Hours + 32 GiB Free Storage) im Free-Tier-Budget — für den Auftragsumfang ausreichend.

2 Architektur

Die Lösung besteht aus drei Hauptbereichen: dem Entwickler mit dem lokalen Repository, GitHub als Code-Hosting und CI/CD-Runner sowie Azure als Laufzeitumgebung. Bei jedem Push auf den main-Branch löst GitHub Actions einen zweistufigen Workflow aus, der das Artefakt baut und auf den App Service deployt.

Architektur · Cloud-Plattform Deployment auf Azure

Auftrag C3 · Deployment LB1 · GIBB Bern

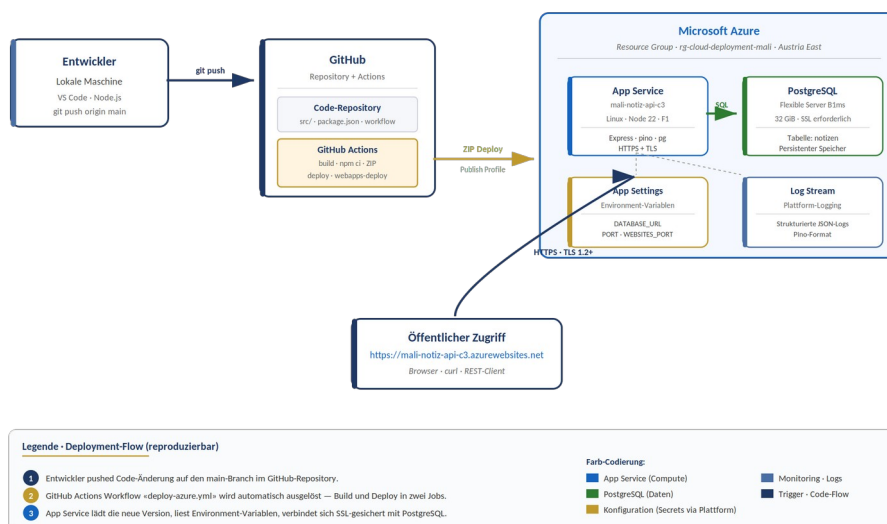


Abbildung 1: Architektur-Diagramm. Für eine bessere Darstellung siehe die beigelegte Datei «architektur.png».

3 Setup-Anleitung

Die folgenden sieben Schritte erlauben einer dritten Person, das Deployment vollständig zu reproduzieren. Voraussetzungen sind ein Azure-Konto (z. B. Azure for Students) sowie ein GitHub-Konto mit dem geklonten Repository.

► Schritt 1 — Resource Group anlegen

Im Azure Portal eine Resource Group `rg-cloud-deployment-mali` in der Region `Austria East` erstellen. Sie dient als Container für alle weiteren Ressourcen.

► Schritt 2 — PostgreSQL Flexible Server anlegen

Über Azure Database for PostgreSQL → flexible servers → Create. Workload-Type `Dev/Test`, Compute-Tier `Burstable Blms`, Storage `32 GiB`. Admin-Login `maliadmin` mit sicherem Passwort. Im Networking-Tab «Public access» aktivieren und die Firewall-Regel `0.0.0.0 – 255.255.255.255` hinzufügen sowie «Allow public access from any Azure service» setzen.

► Schritt 3 — App Service (Web App) anlegen

App Services → Create → Web App. Runtime `Node 22 LTS` auf `Linux`, Pricing Plan `Free F1`. «Basic Authentication» aktiviert lassen, da der Workflow das Publish Profile dafür benötigt.

► Schritt 4 — Environment-Variablen setzen

In App Service → Settings → Environment variables drei Einträge anlegen:

APP SETTINGS	
DATABASE_URL	postgres://maliadmin:<PASSWORT>@pg-notiz-mali-we-2026.postgres.database.azure.com:5432/postgres?sslmode=require
PORT	8080
WEBSITES_PORT	8080

► Schritt 5 — Publish Profile herunterladen

In App Service → Overview → `Download publish profile` klicken. Der vollständige XML-Inhalt der heruntergeladenen `.PublishSettings`-Datei wird im nächsten Schritt benötigt.

► Schritt 6 — GitHub Secret anlegen

Im Repository unter Settings → Secrets and variables → Actions → New repository secret einen Eintrag mit Namen `AZURE_WEBAPP_PUBLISH_PROFILE` anlegen. Wert ist der gesamte XML-Inhalt aus Schritt 5.

► Schritt 7 — Deployment auslösen

Ein Push auf main triggert den Workflow automatisch:

```
BASH
git clone https://github.com/maliswiss/Cloud-Plattform-Deployment-mit-Azure.git
cd Cloud-Plattform-Deployment-mit-Azure
git commit --allow-empty -m "Trigger deployment"
git push origin main
```

Der Workflow läuft unter Actions in ca. 40 Sekunden durch. Die Anwendung ist anschliessend unter `https://mali-notiz-api-c3.azurewebsites.net` erreichbar.

4 Konfigurationsausschnitte

Anwendungs-Konfiguration ausschliesslich über Environment-Variablen

Die Anwendung liest sämtliche umgebungsabhängigen Werte aus `process.env`. Im Code befinden sich keine hartkodierten Secrets oder Verbindungszeichenfolgen — diese werden im Azure App Service als App Settings gepflegt:

```
JAVASCRIPT
const PORT = process.env.PORT || 8080;
const DATABASE_URL = process.env.DATABASE_URL;
```

```
const pool = new Pool({
  connectionString: DATABASE_URL,
  ssl: { rejectUnauthorized: false } // Azure PostgreSQL erfordert SSL
});
```

Strukturierte JSON-Logs via Pino

Logs werden im Pino-Format auf stdout geschrieben und sind im Azure Log Stream als JSON einsehbar — strukturiertes Logging über das Logging-Interface der Plattform, wie in der Aufgabenstellung gefordert:

JAVASCRIPT

```
import pino from 'pino';
import pinoHttp from 'pino-http';

const logger = pino({ level: process.env.LOG_LEVEL || 'info' });
app.use(pinoHttp({ logger }));
```

GitHub Actions Workflow für Auto-Deployment

Der Workflow ist in zwei Jobs aufgeteilt: Build erstellt das ZIP-Artefakt, Deploy lädt es authentifiziert auf den App Service. Manuelle Klicks im Portal sind nicht erforderlich.

YAML

```
on:
  push:
    branches: [main]

env:
  AZURE_WEBAPP_NAME: mali-notiz-api-c3
  NODE_VERSION: '22.x'

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '22.x', cache: 'npm' }
      - run: npm ci --omit=dev
      - run: zip -r release.zip . -x "*.git*" ".github/*" ".env*"
      - uses: actions/upload-artifact@v4
        with: { name: node-app, path: release.zip }

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
      - uses: azure/webapps-deploy@v3
        with:
          app-name: ${ env.AZURE_WEBAPP_NAME }
          publish-profile: ${ secrets.AZURE_WEBAPP_PUBLISH_PROFILE }
          package: release.zip
```

5 Begründung der wichtigsten Entscheidungen

► Plattform-Wahl: Azure App Service statt Self-Hosted

Bewusst wurde eine Managed-Plattform statt einer selbst gehosteten Lösung (z. B. Coolify auf einer VM) gewählt. Der Auftrag fokussiert sich auf Cloud-Deployment, nicht auf Server-Administration. Der Trade-off — Abhängigkeit von den Eigenheiten der Plattform — ist im Studenten-Kontext akzeptabel und reduziert die Setup-Zeit erheblich. TLS-Zertifikate, OS-Patches und Skalierung übernimmt Azure automatisch.

► Datenhaltung: Managed PostgreSQL statt Volume oder SQLite

Eine Managed-Database wurde gewählt, weil sie die Anforderung «Daten überstehen Neustart oder Redeploy» nativ und ohne zusätzliche Konfiguration erfüllt. Ein App-Service-Volume wäre an die Lebensdauer des Containers gebunden; SQLite im Filesystem geht bei jedem Redeploy verloren. PostgreSQL Flexible Server läuft als eigene Ressource ausserhalb des App Service — die Datenbank ist unabhängig vom Lebenszyklus der Anwendung.

► Deployment-Methode: GitHub Actions Workflow

Ein Workflow wurde dem manuellen Portal-Deploy oder dem reinen Git-Push-Trigger des App Service vorgezogen. Erstens ist der Workflow versioniert im Repository und damit nachvollziehbar. Zweitens werden Build und Deploy klar getrennt — der Build ist reproduzierbar (npm ci mit Lock-File), der Deploy ist authentifiziert via Publish Profile. Drittens entspricht es dem in der Bewertung positiv erwähnten «automatisierten Deployment per Pipeline».

6 Reflexion

Imit Containern, CI/CD-Pipelines und Environment-basierter Konfiguration hatte ich vorher schon gearbeitet — bisher allerdings nur lokal oder gegen private Registries. Dieser Auftrag hat mein Verständnis des Übergangs zur öffentlichen Cloud-Plattform mit Managed Database vertieft. Azure App Service nimmt einem TLS, Logging-Interface und persistenten Datenspeicher ab — fordert dafür aber strikte Konventionen: Port-Bindung, Logging-Format auf stdout und Secret-Handling über App Settings müssen exakt zur Plattform passen

► Was gut funktioniert hat

Die Entscheidung, `DATABASE_URL` strikt als Environment-Variable zu behandeln, hat sich von Anfang an ausgezahlt — `.env.example` im Repository und die echte Verbindungszeichenfolge in den Azure App Settings sind sauber getrennt. Pino als Logger war ebenfalls eine gute Wahl: die JSON-Logs erscheinen ohne weitere Konfiguration direkt im Azure Log Stream.

► Was ich rückblickend anders lösen würde

Beim ersten Provisionierungs-Versuch habe ich Switzerland North als Region gewählt. Die Studenten-Subscription hat dies durch eine Region-Policy verweigert — ich musste auf Austria East wechseln. Beim nächsten Mal würde ich mit `az account list-locations` die erlaubten Regionen vorab prüfen, statt iterativ zu provisionieren. Zweitens ist der erste Build gescheitert, weil `package-lock.json` fehlte; ich würde die Lock-Datei künftig direkt bei der Initialisierung anlegen.

► Wichtigste Erkenntnis

Reproduzierbarkeit ist kein Selbstzweck. Die Anforderung, kein manuelles Klicken im Portal, hat im Verlauf der Umsetzung konkrete Vorteile gezeigt — etwa beim Re-Run nach einer Code-Korrektur, die ohne Workflow eine vollständige manuelle Neu-Deployment-Sequenz erfordert hätte.

Hinweis zum Einsatz von KI

In diesem Projekt wurde bei der Erstellung der Dokumentation und einzelner Codeabschnitte teilweise KI (Künstliche Intelligenz) eingesetzt. Sämtliche generierten Inhalte wurden geprüft, angepasst und sind vom Autor inhaltlich nachvollziehbar.